

Part 3: MCP & RAG

[download this slide deck](#)

Overview

Today we will look at two recent popular ways for LLM's to pull contextual data and other digital artifacts. This data is meant to be used in conjunction with user AI sessions.

- ▶ Model Context Protocol (MCP) - MCP is the right choice when the model needs to go beyond answering and start doing. If the task involves interacting with a live system, triggering a process, or producing a real-world outcome, MCP is what makes that possible.
- ▶ Retrieval-Augmented Generation (RAG) - RAG is the right choice when the core job is answering questions from a body of knowledge your model wasn't trained on. If the task is retrieval and synthesis rather than action, RAG is where to start.

Review From last Time

Part 1: Prompting the LLM

We looked at how to prompt and why it is important.

[**download this slide deck**](#)

Part 2: LLM Tokenization

We looked at how the LLM breaks our conversations down into discrete “chunks” known as tokens. We discussed ways to improve AI sessions and increase efficacy of session results.

[**download this slide deck**](#)

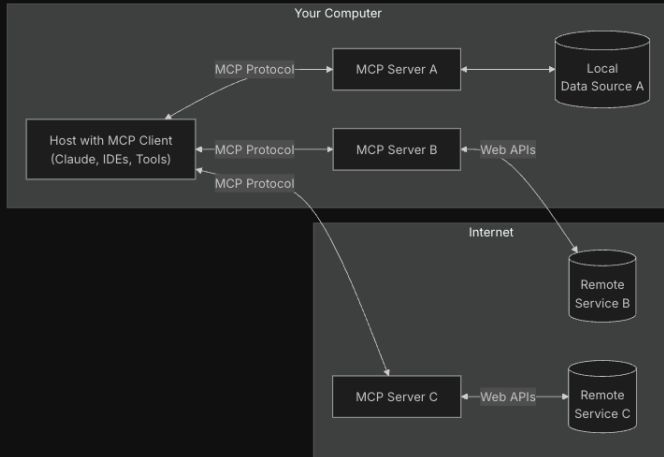
Model Context Protocol: Architecture and Components

Model Context Protocol

- ▶ The MCP creates a standardized, two-way connection for AI applications, allowing LLMs to easily connect with various data sources and tools.
- ▶ MCP builds on existing concepts like tool use and function calling but standardizes them. This reduces the need for custom connections for each new AI model and external system.
- ▶ MCP enables LLMs to use current, real-world data, perform actions, and access specialized features not included in their original training.

General architecture

At its core, MCP follows a client-server architecture where a host application can connect to multiple servers:



MCP Host

The LLM is contained within the MCP host, an AI application or environment such as an AI-powered IDE or conversational AI. This is typically the user's interaction point, where the MCP host uses the LLM to process requests that may require external data or tools.

MCP Client

The MCP client, located within the MCP host, helps the LLM and MCP server communicate with each other.

It translates the LLM's requests for the MCP and converts the MCP's replies for the LLM. It also finds and uses available MCP servers.

MCP Server

The MCP server is the external service that provides context, data, or capabilities to the LLM.

It helps LLMs by connecting to external systems like databases and web services, translating their responses into a format the LLM can understand which helps developers provide diverse functionalities.

MCP Transport Layer

The transport layer uses JSON-RPC 2.0 messages to communicate between the client and server, mainly through two transport methods:

- ▶ Standard input/output (stdio): Works well for local resources, offering fast, synchronous message transmission
- ▶ Server-sent events (SSE): Preferred for remote resources, allowing efficient, real-time data streaming

Retrieval-Augmented Generation: Architecture and Components

Retrieval-Augmented Generation

An approach that improves how language models respond by connecting them to external data sources with a straightforward four-step flow:

- ▶ Step 1: Query: The user submits a question or prompt.
- ▶ Step 2: Retrieve: The system searches an external data source (typically a vector database) for content that's semantically relevant to the query.
- ▶ Step 3: Inject: The retrieved content is added to the model's prompt as context, alongside the original query.
- ▶ Step 4: Generate: The model reads both the query and the retrieved context, then produces a response grounded in that information.

The model only “sees” the most relevant chunks retrieved for that specific query, which keeps responses focused and latency manageable.

Key components of a RAG system

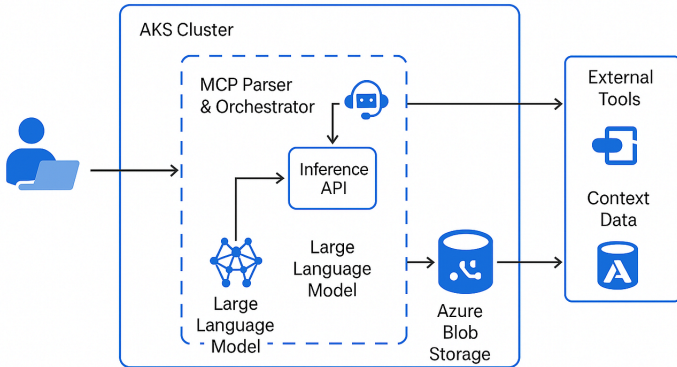
- ▶ **Embeddings:** Text (documents, knowledge base articles, etc.) is converted into numerical representations called embeddings. These capture semantic meaning, so content about similar topics ends up close together in vector space.
- ▶ **Vector database:** The embeddings are stored in a vector database (such as Pinecone, Weaviate, or pgvector). At query time, the user's question is also converted to an embedding, and the database returns the most semantically similar chunks.
- ▶ **Retriever:** This is the logic layer that handles the search, deciding how many chunks to return, applying filters, and ranking results by relevance before passing them to the model.
- ▶ **LLM:** The language model receives the retrieved context alongside the user's query and generates the final response. The quality of the output depends on both the retrieval quality and the model's reasoning ability.

Common RAG use cases

- ▶ Enterprise knowledge assistants: Employees can ask questions against internal wikis, HR policies, or process documentation without digging through folders.
- ▶ Customer support bots: Support agents or end users can get accurate answers drawn directly from product documentation or help articles, reducing hallucination risk.
- ▶ Document Q&A: Legal, finance, or research teams can query long contracts, reports, or research papers and get precise, cited answers.
- ▶ Internal search: Engineering and product teams can surface relevant code documentation, runbooks, or past decisions across large repositories of unstructured text.

Pop Quiz hot Shot

Deploying an LLM with MCP on AKS



Supplemental

My Lab Example

I kept “starting over” by accident because my models did not have a set of static reference materials.

I started creating a PDF document to give my models a “brain” and allow for a common starting point.

Training Resources

Relevant training materials.